

MEW — a self-play data engine for Champions VGC

Design, defended against the prior art in chess, Go and Pokémon

Version 1.0 · 2026-07-25 · ABRA

*Why ABRA generates its own games, why it does so on someone else's engine, and the four places where the chess playbook does **not** transfer to a game with hidden information and a metagame. Every design choice below is stated with the alternative it was chosen over.*

Abstract

ABRA's empirical results are underpowered. On 2026-07-25 the quality-filtered store held 1,061 games, which can detect an edge of no less than 4.3 accuracy points over a coin; resolving a 2-point effect requires ~4,900 games, and human replays accrue at roughly 330 clean games per day. Every null the project has published — JOLTEON, preview roles, CHOMP-EV, WAR, and the rating ceiling — sits inside that blind spot.

MEW removes the constraint for the subset of questions that concern **the game** rather than **people**. It plays the official Showdown Champions engine against itself and writes the outcome in the existing store schema. This paper defends six design decisions against the precedent set by Stockfish, Leela Chess Zero, KataGo, AlphaZero and Metamon, and is explicit about the one thing self-play can never supply.

1. The problem MEW solves, quantified

The power gate added to `engine/eval_harness.py` reports:

Clean games	Smallest detectable edge	Verdict on our current nulls
676 (rated subset)	5.4 pts	cannot distinguish from a coin
1,061 (all clean)	4.3 pts	cannot distinguish from a coin
4,898	2.0 pts	powered
19,623	1.0 pt	powered

A null at n=1,061 is the statement "*no effect larger than 4.3 points was visible*". It is not the statement "*there is no effect*." The project has repeatedly published the first as though it were the second.

Human data closes this in about twelve days for a 2-point effect. Self-play closes it in an afternoon — **for game-level questions only**. That distinction is §6 and it is the most important section here.

2. Precedent: how the strongest engines generate data

2.1 Stockfish NNUE — volume at modest quality beats scarcity at high quality

Stockfish's NNUE evaluation is trained on self-play positions scored at **modest search depth**, typically depth 8–12, with released datasets in the range of **16 billion positions**. It does not attempt to label a small number of positions perfectly. It labels an enormous number adequately.

The transferable lesson is the ratio, not the absolute number. Stockfish concluded that a cheap, consistent label applied at scale beats an expensive label applied rarely. ABRA is at the opposite extreme — 1,061 games, each a real human game of high fidelity, and far too few of them.

2.2 Leela Chess Zero — diversity is a *deliberate parameter*

Leela generates self-play through distributed community compute, and — the detail that matters here — runs with a **policy temperature around 2.25**, deliberately inflating move variety. Left to play its own best line every time, a self-play system converges on a narrow band of positions and the resulting dataset teaches the network only about the states it already reaches.

This is the strongest argument for MEW's first mode. Randomness in self-play is not a defect to be engineered away; it is injected on purpose. §5 develops this.

2.3 KataGo — sample efficiency is a design target

Wu (2019), *Accelerating Self-Play Learning in Go*, is the reference for treating the **cost per useful training sample** as a first-class objective rather than assuming compute is free. ABRA runs on CPU with no GPU, so this is the constraint we actually live under, and it argues for spending self-play budget on positions that resolve uncertainty rather than on more of what we already know.

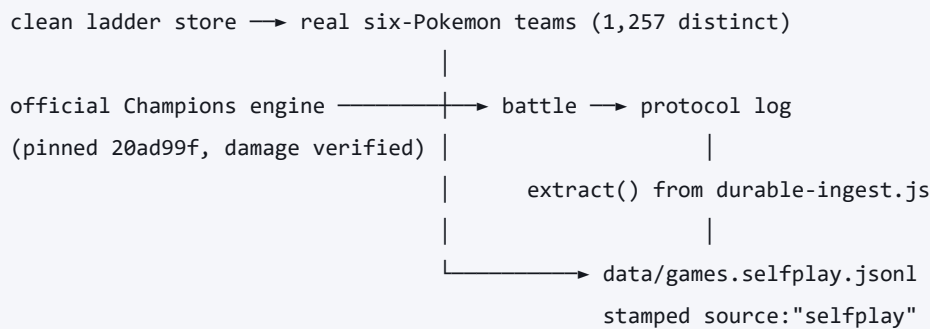
2.4 Metamon — the Pokémon-specific precedent

Metamon (UT-Austin RPL, RLC 2025) trained on **5M human trajectories plus 20M self-play** and reached the top decile of human players. Two things follow. First, the mixture is the established recipe in this exact domain: human data for realism, self-play for volume. Second, the *ratio* is roughly 4:1 in favour of self-play, which is a strong prior for how ABRA should weight its two streams.

2.5 AlphaZero — and why ABRA is not doing this

AlphaZero learns from self-play alone, starting from random weights and no human games. ABRA deliberately does **not** follow this. AlphaZero's setting has no hidden information, no simultaneous moves, and — decisively — **no metagame**. §4.2 explains why that last one breaks the analogy.

3. The design



`engine/mew.js`. Node, because ABRA's standing rule (`ARCHITECTURE-NOTES.md` §1) is that anything touching the battle engine is JavaScript and the statistical core is Python. The Showdown simulator is TypeScript; a Python port would mean maintaining a second copy of the most correctness-critical code in the project.

4. Six decisions, each against its alternative

4.1 The official engine, not ours

Alternative rejected: generate games with `medicham2-browser.js`, which is 117× faster.

Rejected because that engine disagrees with the official simulator by **31.1 percentage points** of win probability and flips the favourite in 3 of 8 matchups (ADR-001). Self-play amplifies engine error rather than averaging it out: every one of a million games inherits the same systematic bias, and a model trained on them learns the bug perfectly. Speed is worthless if the label is wrong.

The official engine's damage was verified against `@smogon/calc` on 31 scenarios, 100% within 2% (`engine/validate_damage_sim.js`), **before** this file was written. That check exists because three of ADR-001's four engine comparisons produced confident wrong numbers from mis-wiring, not from the engine.

4.2 Real teams, not random legal teams — the biggest departure from chess

Alternative rejected: sample random legal teams, as AlphaZero samples from the true game.

This is where the chess analogy breaks and it is worth stating precisely. Chess has one starting position. Every self-play game begins from the same place, so self-play explores the game itself.

VGC has no such thing. The "starting position" is a **team**, drawn from a space of roughly 10^{139} (VGC-Bench), and the tiny inhabited region of that space is the **metagame** — a moving, social object determined by what people are actually playing this week. Uniform sampling from the legal space would generate games between teams no human would ever build, and the resulting matchup statistics would describe a metagame that does not exist.

MEW therefore samples **the 1,257 distinct six-Pokémon teams observed in clean ladder games**. This is the point at which human data and self-play are complements rather than substitutes: the humans supply the distribution over starting positions, the engine supplies unlimited play from them.

A consequence worth stating plainly: MEW inherits whatever bias remains in the team distribution. If the quality filter still admits a bot, MEW will play that bot's team thousands of times. Sampling is therefore

over **distinct** teams, not over games, so a team that appeared 459 times contributes once — exactly the correction that fixed `meta-usage.json`.

4.3 A separate store, enforced in code

Alternative rejected: append to `data/games.ladder.jsonl` and filter on a flag.

Rejected because the project has already been burned by exactly this failure mode. `meta-usage.json` described one script's team as the metagame because a contaminating population was present in a file everything read. A self-play game that ever loses its label is **indistinguishable from a real one**, and the corruption is unrecoverable without re-deriving the whole store.

Output goes to `data/games.selfplay.jsonl`; every record carries `source: "selfplay"` plus the engine commit, format, policy and seed; and `mew.js` **refuses to run** if `--out` resolves to the ladder store. Defence in depth, because the cost of the mistake is asymmetric.

4.4 Reuse the extractor

Alternative rejected: write a self-play-specific record builder.

A second parser is a second definition of what a game *is*, free to drift from the first. Instead a self-play battle log is passed to the **same** `extract()` that parses a downloaded replay. This was verified before `mew.js` was written: a self-play log yields a record with a correct four-Pokémon bring, leads, 11 turns and 8 revealed sets, identical in shape to a ladder game.

The payoff is that every downstream reader — the quality filter, GURU, XATU, PORY, the sanity checks — works on self-play games with no changes at all. This is S1 applied to the data pipeline.

4.5 Random policy first

Alternative rejected: wait for the behaviour-cloned policy before generating anything.

Two reasons, one practical and one substantive.

The practical one: `chooseMove` is the only thing that differs between the two modes. Proving the pipeline with Showdown's own `RandomPlayerAI` de-risks everything else — the engine wiring, the extractor reuse, the provenance, the concurrency — and swapping the policy in later touches one function.

The substantive one is §2.2. Leela runs at temperature 2.25 *on purpose* because a self-play system playing its own best line explores a narrow band of positions. Random play is the extreme of that setting: maximum diversity, minimum realism. It is genuinely the right generator for **matchup structure** — which team beats which, averaged over play — because it does not privilege the lines one particular policy happens to like.

It is genuinely the **wrong** generator for training a value function, for the reason given next.

4.6 Two policies, two purposes — and they are not interchangeable

Generator	Good for	Not valid for
Random	matchup tables, coverage of unusual states, plumbing	training PORY, anything claiming to describe real play
Behaviour-cloned policy	training a value net, realistic trajectories	claiming to be unbiased; it inherits the clone's bias

PORY answers "given this board, what is P(win)". Trained on random games it would answer "given this board, what is P(win) **when both players move at random**", which is not a question anyone has. The literature is unambiguous here — Metamon's human-cloned start, and CICERO's KL-anchoring to human play, both exist because policies trained without a human tether drift into lines that are strong against themselves and alien against people.

So: random now for structure and plumbing; policy before any value-net training. Both are honest provided the output is labelled with which one produced it, which is why `policy` is recorded on every record.

5. What self-play cannot do

Stated as a hard boundary, because the temptation to cross it will be constant once the data is cheap.

Self-play produces **zero** evidence about:

- whether real players tech for the metagame (COUNTERPLAY),
- whether ladder rating predicts anything (the skill-vs-luck study),
- what a human is likely to bring or lead (XATU's priors, CHOMP's belief),
- how the metagame is shifting.

Every one of those is a question about people. A million self-play games move none of them. The human ladder collection is not made redundant by MEW and must keep running — which is the argument for the two streams staying rigorously separate, not merely tidily separate.

There is also a subtler trap. Self-play games are drawn from the team distribution we *observed*, which is three days old and opt-in. Volume does not fix selection bias; it only makes a biased estimate more precise. Sample size and sample validity are different problems, and MEW solves exactly one of them.

6. Validation before use

No self-play data should be trusted before these pass. Listed as the acceptance bar, in the project's own idiom.

1. **Mirror symmetry.** A team against itself must win $50\% \pm$ sampling error. Anything else means a side bias in the harness. Both engines were checked this way in ADR-001 and the check found real problems.
2. **Extractor agreement.** Store-shape invariants (S7) must hold on self-play records exactly as on ladder records: no duplicate ids, `brought  $\subseteq$  six`, `lead  $\subseteq$  brought`, winner is a player.
3. **Determinism.** The same seed must reproduce the same game, or nothing here is reproducible.
4. **Matchup sanity against known ladder results.** Where a matchup is well-sampled in real games, the self-play win rate should not be wildly different. A large divergence is evidence of an engine, team-construction or policy artifact and must be explained before the data is used.

Check 4 is the one that matters most and the one most likely to be skipped.

7. Honest limitations

- **Sets are mostly unknown.** Real replays reveal a mean of 1.38 of four moves, with 69.7% of sets showing no item and 75.5% no ability. `packTeam` fills the gaps from observed-usage priors. Whatever fills them **dominates** the result — this invalidated two engine comparisons in ADR-001 before it was noticed. Self-play games are therefore games between *plausible reconstructions* of real teams, not real teams.
 - **Random play is not play.** See §4.6.
 - **CPU only.** KataGo's efficiency argument is not optional for us.
 - **The team distribution is three days old** and opt-in.
-

8. Sources

- [Stockfish NNUE training datasets](#) — self-play positions at depth 8–12, datasets to 16B positions
- [Leela Chess Zero](#) — distributed self-play; policy temperature ≈ 2.25 for diversity
- Wu, *Accelerating Self-Play Learning in Go* (KataGo), [arXiv:1902.10565](#) — sample efficiency as a design target
- Metamon, *Human-Level Competitive Pokémon via Scalable Offline RL with Transformers*, [arXiv:2504.04395](#) — 5M human + 20M self-play
- VGC-Bench, [arXiv:2506.10326](#) — the $\sim 10^{139}$ team space; exploitability vs generalisation
- Silver et al., *AlphaZero* — self-play from scratch, and the setting whose assumptions do not hold here
- Meta FAIR, *CICERO / piKL* — human-anchored policies; why untethered self-play drifts
- ABRA internal: `docs/ADR-001-use-the-champions-mod.md` , `engine/validate_damage_sim.js` , `docs/STUDY-DESIGN-skill-vs-luck.md`